

claude-windows / 662b86ab-d2e3-4c02-8a0d-ec7a9071aba8

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\662b86ab-d2e3-4c02-8a0d-ec7a9071aba8\subagents\workflows\wf\_9b4c71d0-6a0\agent-a35f22a4d9a2382c8.jsonl`
- SHA-256: `e9f4cc6e4b291d5dbf1d6b11b82417e00f4c0d6163ae561444a63a4bebaa2d81`
- Source modified: `2026-06-21T13:43:02+00:00`
- Imported at: `2026-07-05T16:48:27+00:00`
- project: `wf\_9b4c71d0-6a0`
- session_id: `662b86ab-d2e3-4c02-8a0d-ec7a9071aba8`

Transcript

1. user (2026-06-21T13:40:05.044Z)

Review the COMPUTE_DECOUPLED_SERVING M9 edge-auth change just committed on branch feat/serving-m9-auth (HEAD = 03077c3). Inspect with: git diff origin/master...HEAD and read: backend/api/auth.py (verify_cf_access_token + resolve_tenant + JWKS cache/fetch), backend/main.py (the import-time CORS-from-config + the /api/process tenant resolution), backend/config/models.py (SecurityConfig: edge_auth_enabled/cf_team_domain/cf_access_aud/cors_allow_origins), backend/infrastructure/database.py (create_job owner_id), tests/test_edge_auth.py, pyproject.toml ([dev]+[auth] PyJWT). CONTRACT: DEFAULT-OFF ? security.edge_auth_enabled=False ? resolve_tenant returns None (the local single-user, unchanged) EVEN with a token present; create_job owner_id NULL; CORS default ['*']. When ON, the Cf-Access JWT is REQUIRED + verified against the team JWKS (RS256: sig+exp+iss+aud) so a direct hit to the Cloud Run URL can't spoof the identity header (D9). `jwt` is lazy-imported; the JWKS fetch is injectable for offline tests. Report ONLY real, grounded issues with file:line. Empty findings if a lens finds nothing real.

LENS = DEFAULT-OFF SAFETY & INTEGRATION. (1) resolve_tenant with edge_auth_enabled=False ? None even WITH a header? Trace it. (2) The import-time CORS load: main.py now calls load_app_config() at MODULE_IMPORT (try/except ? ['*']). Does this add a config.json read on every `import backend.main` (tests, CLI) ? any behaviour change, slowdown, or a surprise if a cwd config.json sets cors_allow_origins? Is the middleware fixed at import (can't be reconfigured per-test)? Is that acceptable? (3) create_job gained owner_id as the 6th param ? does ANY existing caller pass positionally in a way that breaks (e.g. a caller passing policy then something)? grep callers. (4) /api/process now takes `request: Request` ? does that change the endpoint's OpenAPI/signature in a way that breaks existing callers or tests? (5) PyJWT in [dev] but the mypy CI lane installs a fixed package list (NOT .[dev]) ? does mypy on backend/api/auth.py fail on the lazy `import jwt`/`from jwt import PyJWKSet` (import-not-found)? Is the lazy import inside the function enough? Find real default-OFF or integration regressions.

2. assistant (2026-06-21T13:40:07.476Z)

I'll review the M9 edge-auth change. Let me start by pulling the latest and examining the diff and relevant files.

3. user (2026-06-21T13:40:09.242Z)

```
03077c3 feat(serving): M9-auth ? Cloudflare Access edge-auth + per-tenant owner_id (default-OFF)
---BRANCH---
feat/serving-m9-auth
---DIFF STAT---
backend/api/auth.py          | 127 +++++
backend/config/models.py     | 14 +++
backend/infrastructure/database.py | 12 +-
backend/main.py              | 29 +----
pyproject.toml               | 5 ++
```

```
tests/test_edge_auth.py | 179 ++++++
6 files changed, 354 insertions(+), 12 deletions(-)
```

4. assistant (2026-06-21T13:40:11.116Z)

Let me read the full diff and the key files.

5. user (2026-06-21T13:40:11.721Z)

```
diff --git a/backend/api/auth.py b/backend/api/auth.py
new file mode 100644
index 0000000..eb187bc
--- /dev/null
+++ b/backend/api/auth.py
@@ -0,0 +1,127 @@
+"""Cloudflare Access edge-auth ? verify the Cf-Access JWT, derive the tenant ``user_id`` (M9, D9).
+
+**DEFAULT-OFF:** when ``security.edge_auth_enabled`` is False the resolver is a strict NO-OP (returns
+``None`` = the local single-user) ? nothing about the existing local path changes. When
+enabled, the
+``Cf-Access-Jwt-Assertion`` header is **required** and **verified** (RS256) against the team
+JWKS ?
+signature + exp + issuer + audience ? so a direct hit to the Cloud Run URL **can't spoof** the
+identity header (the explicit D9 anti-spoof requirement). Returns the verified ``user_id`` (the
+token's ``email``, else ``sub``).
+
+``jwt`` (``PyJWT[crypto]``) is **lazy-imported** so the base runtime works without it (install
+the
+``auth`` extra for hosted/multi-tenant). The JWKS fetch is **injectable**, so the whole
+verifier is
+unit-tested **offline** with an in-test RSA keypair + a fake JWKS ? no network, no Cloudflare.
+"""
+
+from __future__ import annotations
+
+import json
+import time
+import urllib.request
+from typing import Any, Callable, Mapping, Optional
+
+# Cloudflare Access sends the signed assertion in this header (it terminates at the CF edge).
+CF_ACCESS_HEADER = "Cf-Access-Jwt-Assertion"
+_JWKS_PATH = "/cdn-cgi/access/certs"
+_JWKS_TTL_SEC = 600.0 # re-fetch the team JWKS at most every 10 min (keys rotate slowly)
+
+JwksFetcher = Callable[[str], dict]
+
+class EdgeAuthError(Exception):
+    """A required Cf-Access token was missing or failed verification ? the caller returns
+401/403."""
+
+# module-level JWKS cache: team_domain -> (fetched_at, jwks_dict). Production only; tests
+inject.
+_jwks_cache: dict[str, tuple[float, dict]] = {}
+
+def _issuer(team_domain: str) -> str:
+    return team_domain.rstrip("/")
+
+def _default_jwks_fetcher(team_domain: str) -> dict:
```

```

+     """Fetch the team's public JWKS (`<team_domain>/cdn-cgi/access/certs`). Network ? only
the
+     production path reaches here; tests pass an injected fetcher."""
+     url = _issuer(team_domain) + _JWKS_PATH
+     with urllib.request.urlopen(url, timeout=5) as resp: # noqa: S310 (https team domain)
+         data: Any = json.load(resp)
+         if not isinstance(data, dict) or "keys" not in data:
+             raise EdgeAuthError(f"unexpected JWKS shape from {url}")
+     return data
+
+
+def _get_jwks(team_domain: str, fetcher: Optional[JwksFetcher]) -> dict:
+     """Return the JWKS for `team_domain`, cached with a TTL. An injected `fetcher` (tests)
+     bypasses the cache so each test is deterministic."""
+     if fetcher is not None:
+         return fetcher(team_domain)
+     now = time.time()
+     cached = _jwks_cache.get(team_domain)
+     if cached and (now - cached[0]) < _JWKS_TTL_SEC:
+         return cached[1]
+     jwks = _default_jwks_fetcher(team_domain)
+     _jwks_cache[team_domain] = (now, jwks)
+     return jwks
+
+
+def verify_cf_access_token(token: str, *, team_domain: str, aud: str, jwks: dict) -> str:
+     """Verify a Cf-Access JWT against `jwks` (RS256; checks sig + exp + issuer + audience)
and
+     return the user_id (`email` else `sub`). Raises `EdgeAuthError` on ANY failure."""
+     try:
+         import jwt
+         from jwt import PyJWKSet
+     except Exception as exc: # pragma: no cover - only when the auth extra isn't installed
+         raise EdgeAuthError(
+             "PyJWT[crypto] is required for edge auth ? install the 'auth' extra "
+             "(pip install -e .[auth]) on a host with edge_auth_enabled."
+         ) from exc
+
+     try:
+         kid = jwt.get_unverified_header(token).get("kid")
+         if not kid:
+             raise EdgeAuthError("token header has no kid")
+         signing_key = PyJWKSet.from_dict(jwks)[kid]
+         claims = jwt.decode(
+             token, signing_key.key, algorithms=["RS256"],
+             audience=aud, issuer=_issuer(team_domain),
+             options={"require": ["exp", "iss", "aud"]},
+         )
+     except EdgeAuthError:
+         raise
+     except KeyError as exc: # kid not in the JWKS
+         raise EdgeAuthError(f"no JWKS key for kid {exc}") from exc
+     except Exception as exc: # jwt.* errors: bad sig / expired / wrong aud|iss / malformed
+         raise EdgeAuthError(f"Cf-Access token verification failed: {exc}") from exc
+
+     user = claims.get("email") or claims.get("sub")
+     if not user:
+         raise EdgeAuthError("verified token carries no email/sub claim")
+     return str(user)
+
+
+def resolve_tenant(headers: Mapping[str, str], config: Any, *,
+     jwks_fetcher: Optional[JwksFetcher] = None) -> Optional[str]:
+     """Resolve the tenant `user_id` for a request, or `None` when edge auth is OFF.

```

```

+     DEFAULT-OFF (``security.edge_auth_enabled=False``) ? ``None`` (the local single-user;
unchanged).
+     Enabled ? require + verify the Cf-Access header; raise ``EdgeAuthError`` (? 401/403) on any
miss.
+     ``jwks_fetcher`` is injectable for offline tests.
+     """
+     sec = getattr(config, "security", None)
+     if not sec or not getattr(sec, "edge_auth_enabled", False):
+         return None # DEFAULT-OFF ? no tenant identity, byte-identical to today
+
+     team_domain = getattr(sec, "cf_team_domain", "") or ""
+     aud = getattr(sec, "cf_access_aud", "") or ""
+     if not team_domain or not aud:
+         raise EdgeAuthError(
+             "edge_auth_enabled but security.cf_team_domain / cf_access_aud are not configured")
+
+     # Headers are case-insensitive over HTTP; Starlette's Headers handles that, but support a
plain
+     # dict too (tests) by trying both casings.
+     token = headers.get(CF_ACCESS_HEADER) or headers.get(CF_ACCESS_HEADER.lower())
+     if not token:
+         raise EdgeAuthError(f"missing {CF_ACCESS_HEADER} header (request did not pass
Cloudflare Access)")
+
+     jwks = _get_jwks(team_domain, jwks_fetcher)
+     return verify_cf_access_token(token, team_domain=team_domain, aud=aud, jwks=jwks)
diff --git a/backend/config/models.py b/backend/config/models.py
index 71cdc0e..4f27623 100644
--- a/backend/config/models.py
+++ b/backend/config/models.py
@@ -293,6 +293,20 @@ class SecurityConfig(BaseModel):
     max_part_mb: int = 95
     allowed_upload_extensions: List[str] = ["mp4", "mov"]

+     # --- M9 edge auth (Cloudflare Access, D9) ---
+     # DEFAULT-OFF: edge_auth_enabled=False ? no JWT is required/verified (the local single-user
+     # path is unchanged). When True, /api/process requires + verifies the `Cf-Access-Jwt-
Assertion`
+     # header against the team JWKS (so a direct hit to the Cloud Run URL can't spoof the
identity
+     # header) and tags the job with the verified user_id (owner_id). The team domain (e.g.
+     # ``https://<team>.cloudflareaccess.com``) supplies the JWKS + issuer; the AUD is the CF
Access
+     # application's audience tag. NO secrets here ? these are public identifiers.
+     edge_auth_enabled: bool = False
+     cf_team_domain: str = "" # e.g. https://<team>.cloudflareaccess.com (issuer + /certs
JWKS)
+     cf_access_aud: str = "" # the CF Access application AUD tag the token must carry
+     # CORS allow-origins. Default ["*"] keeps the local web UI working; tighten to the hosted
+     # origin(s) in multi-tenant mode. (credentials stay OFF ? Cf-Access is a header, not a
cookie.)
+     cors_allow_origins: List[str] = ["*"]
+
class StorageConfig(BaseModel):
    """Pluggable storage backend (docs/DECOUPLED_COMPUTE/). Default ``local`` = today's
diff --git a/backend/infrastructure/database.py b/backend/infrastructure/database.py
index 6dede93..4e0ac4a 100644
--- a/backend/infrastructure/database.py
+++ b/backend/infrastructure/database.py
@@ -538,15 +538,17 @@ class Database:
     def create_job(self, job_id: str, video_path: str, segmenter: Optional[str] = None,
                    player_pool: Optional[List[str]] = None,
                    max_frames: Optional[int] = None,
                    policy: Optional[Dict[str, Any]] = None) -> bool:

```

```

+         policy: Optional[Dict[str, Any]] = None,
+         owner_id: Optional[str] = None) -> bool:
+         """Insert a new job in state 'queued'. ``policy`` is an optional JSON ExecutionPolicy
-         (EXECUTION_POLICY.md) the worker honours; None ? the worker's default policy."""
+         (EXECUTION_POLICY.md) the worker honours; None ? the worker's default policy.
+         ``owner_id``
+         (M9, D9) is the verified tenant from edge auth ? NULL for the local single-user
+         (default)."""
+         return self._execute_write(
-         "INSERT INTO jobs (id, video_path, segmenter, player_pool, max_frames, status,
policy) "
-         "VALUES (?, ?, ?, ?, ?, 'queued', ?)",
+         "INSERT INTO jobs (id, video_path, segmenter, player_pool, max_frames, status,
policy, "
+         "owner_id) VALUES (?, ?, ?, ?, ?, 'queued', ?, ?)",
+         (job_id, video_path, segmenter,
+         json.dumps(player_pool) if player_pool is not None else None,
+         max_frames, json.dumps(policy) if policy is not None else None),
-         max_frames, json.dumps(policy) if policy is not None else None, owner_id),
+         f"Failed to create job {job_id}",
        )

diff --git a/backend/main.py b/backend/main.py
index ef1588f..c0be6b1 100644
--- a/backend/main.py
+++ b/backend/main.py
@@ -16,7 +16,7 @@ logging.basicConfig(
    datefmt='%H:%M:%S'
)
logger = logging.getLogger(__name__)
-from fastapi import FastAPI, HTTPException
+from fastapi import FastAPI, HTTPException, Request
+from fastapi.middleware.cors import CORSMiddleware
+from pydantic import BaseModel
+from typing import List, Dict, Any, Optional
@@ -40,15 +40,22 @@ from backend.results import Failure # standardized error/result contract
+from backend.reporting import emit_indexer_report
+from backend import storage # M2: URI-aware input acquisition (local = identity passthrough)
+from backend import billing # M8: per-job cost ledger (USD internal / INR display)
+from backend.api import auth as edge_auth # M9: Cloudflare Access edge-auth (default-OFF)

app = FastAPI(title="Sports Highlight Indexer API")

-# Enable CORS for the local web interface. allow_origins='' with allow_credentials=True is
-# rejected by browsers per the fetch spec and is an unsafe default to carry into the
-# auth/tenancy work; this tool uses no cookies, so credentials stay off.
+# CORS for the web UI. allow_origins='' with allow_credentials=True is rejected by browsers
per the
+# fetch spec + is unsafe to carry into multi-tenant; this tool uses no cookies, so credentials
stay
+# off. M9: the origin list is configurable (security.cors_allow_origins, default [""]) so a
hosted
+# deploy can lock it to its origin. Read at import (a cheap, torch-free config load; defaults
when no
+# config.json) ? the middleware is fixed at startup.
+try:
+    _cors_origins = load_app_config().security.cors_allow_origins
+except Exception:
+    _cors_origins = [""]
+    app.add_middleware(
+        CORSMiddleware,
-        allow_origins=[""],
+        allow_origins=_cors_origins,
+        allow_credentials=False,
+        allow_methods=[""],

```

```

    allow_headers=["*"],
@@ -266,7 +273,7 @@ def _execute_processing(req: ProcessRequest, progress=None) -> Dict[str,
Any]:

@app.post("/api/process", status_code=202)
-def process_video(req: ProcessRequest):
+def process_video(req: ProcessRequest, request: Request):
    """Submit a processing job (202 + job_id). Poll GET /api/jobs/{job_id} for state.

    Fast-fail checks (bad path, missing file, unknown segmenter) happen here so the
@@ -274,6 +281,14 @@ def process_video(req: ProcessRequest):
    runs on the worker (DEFERRED_HARDENING_PLAN #16). The row is created 'queued'; the
    worker drains it via `claim_due_job` (EXECUTION_POLICY.md P3 self-scheduling loop).
    """
+
+ # M9 (D9): edge-auth tenant resolution. DEFAULT-OFF ? owner_id None (the local single-user,
+ # unchanged). When edge_auth_enabled, the Cf-Access JWT is required + verified (a missing
or
+ # spoofed identity header fails here with 401, before any work) and tags the job's
owner_id.
+ try:
+     owner_id = edge_auth.resolve_tenant(request.headers, global_config)
+ except edge_auth.EdgeAuthError as e:
+     raise HTTPException(status_code=401, detail=f"edge auth: {e}") from e
+
    allowed_root = getattr(getattr(global_config, "security", None), "allowed_video_root",
None)
    try:
        validate_input_path(req.video_path, allowed_root=allowed_root)
@@ -299,7 +314,7 @@ def process_video(req: ProcessRequest):

    job_id = uuid.uuid4().hex
    if not db_instance.create_job(job_id, req.video_path, req.segmenter,
-                                req.player_pool, req.max_frames):
+                                req.player_pool, req.max_frames, owner_id=owner_id):
        raise HTTPException(status_code=500, detail="Failed to persist job record.")
    _get_executor().submit(_drain_due_jobs)
    return JSONResponse(
diff --git a/pyproject.toml b/pyproject.toml
index ec11f3f..282bc83 100644
--- a/pyproject.toml
+++ b/pyproject.toml
@@ -44,8 +44,13 @@ dev = [
    "httpx",          # fastapi TestClient transport
    "ruff",
    "mypy",
+    "PyJWT[crypto]>=2.8", # M9 edge-auth: Cf-Access JWT verify (RS256) ? tests + CI
]

+# M9 edge-auth (Cloudflare Access). DEFAULT-OFF: only needed when
security.edge_auth_enabled=True.
+# `jwt` is lazy-imported, so the base runtime works without this; install it for hosted/multi-
tenant.
+auth = ["PyJWT[crypto]>=2.8"]
+
+ # Per-video observability reports (SOFT dependency: the pipeline degrades to a
+ # no-op if absent). Pinned to its git tag per BACKLOG. The private repo is
+ # fetched over SSH; if you lack access, simply omit this group ? reporting
diff --git a/tests/test_edge_auth.py b/tests/test_edge_auth.py
new file mode 100644
index 0000000..f7be7d25
--- /dev/null
+++ b/tests/test_edge_auth.py
@@ -0,0 +1,179 @@
+"""Cloudflare Access edge-auth (COMPUTE_DECOUPLED_SERVING M9, D9).

```

```

+
+Fully OFFLINE: an in-test RSA keypair signs Cf-Access JWTs + builds a fake JWKS, so the
verifier +
+the tenant resolver are exercised with no network and no Cloudflare. Cardinal invariant: with
+`edge_auth_enabled=False` the resolver is a strict NO-OP (returns None = today's local
single-user)
+EVEN when a token is present.
+"""
+import json
+import time
+
+import pytest
+
+pytest.importorskip("jwt") # PyJWT[crypto] (in the [dev] extra); skip cleanly where absent
+import jwt # noqa: E402
+from jwt.algorithms import RSAAlgorithm # noqa: E402
+from cryptography.hazmat.primitives.asymmetric import rsa # noqa: E402
+
+from backend.api import auth as edge_auth # noqa: E402
+from backend.config import AppConfig # noqa: E402
+
+TEAM = "https://team.cloudflareaccess.com"
+AUD = "test-aud-tag"
+KID = "test-kid"
+
+
+@pytest.fixture(scope="module")
+def keypair():
+    """An RSA private key + a JWKS carrying its public key (kid=test-kid)."""
+    key = rsa.generate_private_key(public_exponent=65537, key_size=2048)
+    jwk = json.loads(RSAAlgorithm.to_jwk(key.public_key()))
+    jwk.update({"kid": KID, "alg": "RS256", "use": "sig"})
+    return key, {"keys": [jwk]}
+
+
+def _token(key, *, email="u@example.com", aud=AUD, iss=TEAM, exp_delta=3600, kid=KID,
sub=None):
+    now = int(time.time())
+    claims = {"aud": aud, "iss": iss, "exp": now + exp_delta, "iat": now}
+    if email:
+        claims["email"] = email
+    if sub:
+        claims["sub"] = sub
+    return jwt.encode(claims, key, algorithm="RS256", headers={"kid": kid})
+
+
+def _cfg(enabled, *, team=TEAM, aud=AUD):
+    return AppConfig.model_validate({"security": {
+        "edge_auth_enabled": enabled, "cf_team_domain": team, "cf_access_aud": aud}})
+
+
+# ----- verifier (accept)
+
+def test_valid_token_returns_email(keypair):
+    key, jwks = keypair
+    assert edge_auth.verify_cf_access_token(
+        _token(key), team_domain=TEAM, aud=AUD, jwks=jwks) == "u@example.com"
+
+
+def test_falls_back_to_sub_when_no_email(keypair):
+    key, jwks = keypair
+    tok = _token(key, email=None, sub="user-123")
+    assert edge_auth.verify_cf_access_token(tok, team_domain=TEAM, aud=AUD, jwks=jwks) ==
"user-123"
+
+

```

```

+
+## ----- verifier (reject paths)
+
+@pytest.mark.parametrize("kw", [
+    {"exp_delta": -10},          # expired
+    {"aud": "wrong-aud"},        # wrong audience (not this CF app)
+    {"iss": "https://evil.example"}, # wrong issuer
+    {"kid": "unknown-kid"},      # kid not in the JWKS
+    {"email": None, "sub": None}, # no identity claim
+])
+def test_invalid_tokens_rejected(keypair, kw):
+    key, jwks = keypair
+    with pytest.raises(edge_auth.EdgeAuthError):
+        edge_auth.verify_cf_access_token(_token(key, **kw), team_domain=TEAM, aud=AUD,
+jwks=jwks)
+
+
+def test_bad_signature_rejected(keypair):
+    """A token signed by a DIFFERENT key (but advertising kid=test-kid) must fail signature
+check."""
+    _, jwks = keypair
+    other = rsa.generate_private_key(public_exponent=65537, key_size=2048)
+    with pytest.raises(edge_auth.EdgeAuthError):
+        edge_auth.verify_cf_access_token(_token(other), team_domain=TEAM, aud=AUD, jwks=jwks)
+
+
+## ----- resolve_tenant (default-
OFF)
+
+def test_resolve_tenant_off_is_noop_even_with_token(keypair):
+    """DEFAULT-OFF: edge_auth.enabled=False ? None, even when a (valid) token is present."""
+    key, _ = keypair
+    assert edge_auth.resolve_tenant(
+        {edge_auth.CF_ACCESS_HEADER: _token(key)}, _cfg(False)) is None
+
+
+def test_resolve_tenant_off_ignores_missing_header():
+    assert edge_auth.resolve_tenant({}, _cfg(False)) is None
+
+
+## ----- resolve_tenant (enabled)
+
+def test_resolve_tenant_on_valid(keypair):
+    key, jwks = keypair
+    user = edge_auth.resolve_tenant(
+        {edge_auth.CF_ACCESS_HEADER: _token(key)}, _cfg(True), jwks_fetcher=lambda td: jwks)
+    assert user == "u@example.com"
+
+
+def test_resolve_tenant_on_missing_header_raises(keypair):
+    _, jwks = keypair
+    with pytest.raises(edge_auth.EdgeAuthError):
+        edge_auth.resolve_tenant({}, _cfg(True), jwks_fetcher=lambda td: jwks)
+
+
+def test_resolve_tenant_enabled_but_unconfigured_raises(keypair):
+    key, _ = keypair
+    with pytest.raises(edge_auth.EdgeAuthError):
+        edge_auth.resolve_tenant({edge_auth.CF_ACCESS_HEADER: _token(key)}, _cfg(True, team="",
+aud=""))
+
+
+def test_resolve_tenant_header_case_insensitive(keypair):
+    key, jwks = keypair
+    user = edge_auth.resolve_tenant(

```



```

+         {edge_auth.CF_ACCESS_HEADER.lower(): _token(key)}, _cfg(True), jwks_fetcher=lambda td:
jwks)
+     assert user == "u@example.com"
+
+
+## ----- /api/process wiring (owner_id tag)
+
+pytest.importorskip("httpx") # fastapi TestClient transport
+
+
+def _client(tmp_path, monkeypatch, cfg):
+     from fastapi.testclient import TestClient
+
+     import backend.main as m
+     from backend.infrastructure.database import Database
+
+     db = Database(str(tmp_path / "t.db"))
+     monkeypatch.setattr(m, "db_instance", db)
+     monkeypatch.setattr(m, "global_config", cfg)
+
+     class _Noop: # don't drain the job (we only assert the created row's owner_id)
+         def submit(self, fn, *a, **k):
+             return None
+
+     monkeypatch.setattr(m, "_get_executor", lambda: _Noop())
+     edge_auth._jwks_cache.clear()
+     return TestClient(m.app), db, m
+
+
+def test_process_default_off_tags_no_owner(tmp_path, monkeypatch):
+     """DEFAULT-OFF: /api/process needs no token and the job's owner_id is NULL (unchanged)."""
+     client, db, _m = _client(tmp_path, monkeypatch, AppConfig()) # edge_auth_enabled=False
+     v = tmp_path / "x.mp4"
+     v.write_bytes(b"x")
+     r = client.post("/api/process", json={"video_path": str(v)})
+     assert r.status_code == 202
+     assert db.get_job(r.json()["job_id"])["owner_id"] is None
+
+
+def test_process_edge_auth_missing_token_401(tmp_path, monkeypatch):
+     client, _db, _m = _client(tmp_path, monkeypatch, _cfg(True))
+     v = tmp_path / "x.mp4"
+     v.write_bytes(b"x")
+     r = client.post("/api/process", json={"video_path": str(v)}) # no Cf-Access header
+     assert r.status_code == 401
+
+
+def test_process_edge_auth_valid_token_sets_owner(tmp_path, monkeypatch, keypair):
+     key, jwks = keypair
+     client, db, m = _client(tmp_path, monkeypatch, _cfg(True))
+     monkeypatch.setattr(m.edge_auth, "_default_jwks_fetcher", lambda td: jwks)
+     v = tmp_path / "x.mp4"
+     v.write_bytes(b"x")
+     r = client.post("/api/process", json={"video_path": str(v)},
+                     headers={edge_auth.CF_ACCESS_HEADER: _token(key)})
+     assert r.status_code == 202
+     assert db.get_job(r.json()["job_id"])["owner_id"] == "u@example.com"

```

6. assistant (2026-06-21T13:40:17.027Z)

Now let me examine the key files in detail. Let me look at the `owner\_id` column in the schema, the `create\_job` callers, and the config loading.

7. user (2026-06-21T13:40:18.177Z)

```

tests\test_async_jobs.py:151: db.create_job(jid, str(vid_file2))
tests\test_async_jobs.py:316: db.create_job("q1", "/v.mp4")
tests\test_async_jobs.py:317: db.create_job("r1", "/v.mp4")
tests\test_async_jobs.py:319: db.create_job("d1", "/v.mp4")
tests\test_billing.py:84: db.create_job("j1", "v.mp4")
tests\test_billing.py:100: db.create_job("j1", "v.mp4")
tests\test_billing.py:108: db.create_job("j1", "v.mp4")
tests\test_billing.py:123: db.create_job("j1", "v.mp4")
tests\test_billing.py:141: db.create_job(jid, "v.mp4")
tests\test_billing.py:159: db.create_job("j1", "v.mp4")
tests\test_billing.py:171: db.create_job("j1", "v.mp4")
tests\test_billing.py:186: db.create_job("j1", "v.mp4")
tests\test_api_endpoints.py:591: db.create_job("j1", "v.mp4")
tests\test_api_endpoints.py:606: db.create_job("j1", "v.mp4")
tests\test_api_endpoints.py:620: db.create_job(jid, "v.mp4")
tests\test_database.py:485: ok = db.create_job("job-99", "/tmp/v.mp4")
backend\main.py:316: if not db_instance.create_job(job_id, req.video_path, req.segmenter,
backend\infrastructure\database.py:538: def create_job(self, job_id: str, video_path: str,
segmenter: Optional[str] = None,
tests\test_job_self_scheduling.py:17: assert db.create_job("j1", "/v.mp4", policy=POLICY)
tests\test_job_self_scheduling.py:23: db.create_job("j0", "/v.mp4")
tests\test_job_self_scheduling.py:29: db.create_job("j1", "/v.mp4")
tests\test_job_self_scheduling.py:36: db.create_job("j1", "/v.mp4")
tests\test_job_self_scheduling.py:49: db.create_job("j1", "/v.mp4", policy=POLICY)
tests\test_job_self_scheduling.py:58: db.create_job("a", "/a.mp4")
tests\test_job_self_scheduling.py:59: db.create_job("b", "/b.mp4")
tests\test_job_self_scheduling.py:67: db.create_job("j1", "/v.mp4")
tests\test_job_self_scheduling.py:75: db.create_job("j1", "/v.mp4")
tests\test_job_self_scheduling.py:83: db.create_job("j1", "/v.mp4")
tests\test_job_worker_loop.py:104: db.create_job("a", str(a), segmenter="x")
tests\test_job_worker_loop.py:105: db.create_job("b", str(b), segmenter="x")
tests\test_job_worker_loop.py:167: db.create_job("j1", "/v.mp4")
tests\test_job_worker_loop.py:191: db.create_job("a", "/a.mp4")
tests\test_job_worker_loop.py:192: db.create_job("b", "/b.mp4")
tests\test_job_worker_loop.py:207: db.create_job("q", "/v.mp4") # queued, not waiting
tests\test_job_worker_loop.py:213: db.create_job("j", "/v.mp4")
tests\test_job_worker_loop.py:220: db.create_job("j", "/v.mp4")
tests\test_job_worker_loop.py:228: db.create_job("far", "/a.mp4")
tests\test_job_worker_loop.py:229: db.create_job("near", "/b.mp4")
tests\test_job_worker_loop.py:251: db.create_job("j", str(vid_file), segmenter="x")
tests\test_job_worker_loop.py:266: db.create_job("j", "/v.mp4")
tests\test_job_worker_module_seam.py:49: db.create_job("j1", "/v.mp4")

```

8. user (2026-06-21T13:40:18.896Z)

```

backend\api\job_worker.py:114: claimed row (carries ``owner_id``); ``result`` is the pipeline
output.
backend\api\job_worker.py:125: owner_id=job.get("owner_id"),
margin=billing_cfg.margin,
backend\config\models.py:300: # header) and tags the job with the verified user_id
(owner_id). The team domain (e.g.
backend\infrastructure\database.py:300: ("owner_id", "TEXT"), # who
pays (the non-owner user / tenant)
backend\infrastructure\database.py:313: cursor.execute("CREATE INDEX IF NOT EXISTS
idx_jobs_owner ON jobs(owner_id) "
backend\infrastructure\database.py:314: "WHERE owner_id IS NOT NULL")
backend\infrastructure\database.py:542: owner_id: Optional[str] = None) ->
bool:
backend\infrastructure\database.py:544: (EXECUTION_POLICY.md) the worker honours; None ?
the worker's default policy. ``owner_id``
backend\infrastructure\database.py:548: "owner_id) VALUES (?, ?, ?, ?, ?, 'queued',
?, ?)",
backend\infrastructure\database.py:551: max_frames, json.dumps(policy) if policy is
not None else None, owner_id),
backend\infrastructure\database.py:626: owner_id: Optional[str] = None,

```

```

margin: float = 0.0) -> Optional[Dict[str, Any]]:
backend\infrastructure\database.py:650:         "UPDATE jobs SET owner_id=?,
compute_backend=?, gpu_type=?, gpu_active_seconds=?, "
backend\infrastructure\database.py:653:         (owner_id, compute_backend, gpu_type,
usage.get(billing.GPU_SECONDS),
backend\infrastructure\database.py:667:         "SELECT id, video_id, owner_id,
compute_backend, gpu_type, gpu_active_seconds, "
backend\main.py:284:     # M9 (D9): edge-auth tenant resolution. DEFAULT-OFF ? owner_id None (the
local single-user,
backend\main.py:286:     # spoofed identity header fails here with 401, before any work) and tags
the job's owner_id.
backend\main.py:288:         owner_id = edge_auth.resolve_tenant(request.headers, global_config)
backend\main.py:317:         req.player_pool, req.max_frames,
owner_id=owner_id):

```

9. assistant (2026-06-21T13:40:22.694Z)

The `owner\_id` column already exists in the schema (line 300), pre-dating this change (used by billing M8). Good ? no migration needed. All `create\_job` callers use keyword `policy=` or positional up to the 5th arg, none pass a 6th positional, so adding `owner\_id` as 6th keyword-only-in-practice param is safe. Let me verify the schema column and migration ladder, and check the `load\_app\_config` import in main.py.

10. user (2026-06-21T13:40:24.122Z)

```

280         CREATE UNIQUE INDEX IF NOT EXISTS idx_user_models_user_version
281             ON user_models(user_id, version)
282     """
283     # At most ONE active model per user: a partial-unique index over the active
rows.
284     # (SQLite treats NULL user_id rows as distinct, so the local install's single
285     # active row is still guarded ? there is one local user.)
286     cursor.execute("""
287         CREATE UNIQUE INDEX IF NOT EXISTS idx_user_models_one_active
288             ON user_models(user_id) WHERE is_active = 1
289     """)
290     cursor.execute("PRAGMA user_version = 5")
291
292     def _migrate_to_v6(self, cursor):
293         # v6: per-job COST LEDGER (COMPUTE_DECOUPLED_SERVING M8, D8). NULLABLE cost
columns on
294         # `jobs` (NULL = no cost recorded = byte-identical to today; nothing records
until
295         # billing.enabled + a real pricebook) + a VERSIONED `pricebook` table. Cost is
stored in
296         # USD (matches GCP billing = one source of truth); INR is a display conversion
at a
297         # configurable FX rate. ALTER (not recreate) so v1-v5 job rows survive; ADD
COLUMNS are
298         # idempotent so an interrupted upgrade can re-run them. Additive ONLY ? no
served change.
299         for col, typ in (
300             ("owner_id", "TEXT"),           # who pays (the non-owner user / tenant)
301             ("compute_backend", "TEXT"),     # local_gpu | cloud_run | ...
302             ("gpu_type", "TEXT"),           # L4 | T4 | ... (selects the per-second
GPU rate)
303             ("gpu_active_seconds", "REAL"),
304             ("wall_seconds", "REAL"),
305             ("bytes_out", "INTEGER"),       # egress
306             ("gemini_cost_usd", "REAL"),    # provider-reported, already USD
307             ("pricebook_version", "TEXT"),
308             ("cost_usd", "REAL"),           # computed total (USD = source of truth)
309             ("cost_breakdown_json", "TEXT"), # {gpu_seconds: ..., egress_gb: ...,
gemini_usd: ...}
310         ):

```

```

311         self._add_column_idempotent(cursor, f"ALTER TABLE jobs ADD COLUMN {col}
{typ}")
312         # Partial index keeps the NULL=no-cost fast path churn-free while making per-
owner billing cheap.
313         cursor.execute("CREATE INDEX IF NOT EXISTS idx_jobs_owner ON jobs(owner_id) "
314                        "WHERE owner_id IS NOT NULL")
315         # Versioned pricebook: one row per (version, resource, gpu_type). Rates change +
differ by
316         # gpu_type ? re-version (never mutate a shipped version, so a recorded cost
stays auditable).
317         cursor.execute("""
318             CREATE TABLE IF NOT EXISTS pricebook (
319                 version TEXT NOT NULL,
320                 resource TEXT NOT NULL,
321                 gpu_type TEXT NOT NULL DEFAULT '',
322                 unit_price_usd REAL NOT NULL DEFAULT 0.0,
323                 currency TEXT NOT NULL DEFAULT 'USD',
324                 effective_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
325             )
326         """)
327         cursor.execute("CREATE UNIQUE INDEX IF NOT EXISTS idx_pricebook_key "
328                        "ON pricebook(version, resource, gpu_type)")
329         # Seed a PLACEHOLDER version with every rate 0.0 ? cost_usd computes to 0
(nothing
330         # user-facing changes). The owner INSERTs a real, calibrated version (real GCP
$/GPU-s,
331         # egress $/GB) and flips billing.pricebook_version to it ? never overwrite this
seed.
332         for resource, gpu_type in (("gpu_seconds", "L4"), ("gpu_seconds", "T4"),
333                                    ("wall_seconds", ""), ("egress_gb", "")):
334             cursor.execute(
335                 "INSERT OR IGNORE INTO pricebook (version, resource, gpu_type,
unit_price_usd, "
336                 "currency) VALUES ('placeholder-v0', ?, ?, 0.0, 'USD')", (resource,
gpu_type))
337         cursor.execute("PRAGMA user_version = 6")
338
339         def add_video(self, video_id: str, filepath: str, status: str, validation_results:
List[Dict[str, Any]]) -> bool:

```

11. user (2026-06-21T13:40:24.934Z)

```

25     from fastapi.responses import FileResponse, HTMLResponse, JSONResponse
26     from backend.pipeline.validators.base import registry
27     from backend.infrastructure.database import Database
28     from backend.pipeline.segmenters.base import segmenter_registry
29     from backend.pipeline.stitcher.compiler import VideoCompiler
30     from backend.utils.hashing import compute_video_id # canonical file-identity hashing
31     from backend.utils.paths import resolve_under_root, safe_output_filename,
validate_input_path
32     from backend.config import ( # new typed config
33         load_config as load_app_config,
34         AppConfig,
35         StitchingConfig,
36         enforce_startup_checks,
37         StartupCheckError,
38     )
39     from backend.results import Failure # standardized error/result contract
40     from backend.reporting import emit_indexer_report
41     from backend import storage # M2: URI-aware input acquisition (local = identity
passthrough)
42     from backend import billing # M8: per-job cost ledger (USD internal / INR display)
43     from backend.api import auth as edge_auth # M9: Cloudflare Access edge-auth (default-
OFF)
44

```

```

45     app = FastAPI(title="Sports Highlight Indexer API")
46
47     # CORS for the web UI. allow_origins='' with allow_credentials=True is rejected by
48     # browsers per the
49     # fetch spec + is unsafe to carry into multi-tenant; this tool uses no cookies, so
50     # credentials stay
51     # off. M9: the origin list is configurable (security.cors_allow_origins, default ["*"])
52     # so a hosted
53     # deploy can lock it to its origin. Read at import (a cheap, torch-free config load;
54     # defaults when no
55     # config.json) ? the middleware is fixed at startup.
56     try:
57         _cors_origins = load_app_config().security.cors_allow_origins
58     except Exception:
59         _cors_origins = ["*"]
60     app.add_middleware(
61         CORSMiddleware,
62         allow_origins=_cors_origins,
63         allow_credentials=False,
64         allow_methods=["*"],
65         allow_headers=["*"],
66     )
67
68     # Serves static frontend files directly (now under api/ per approved structure)
69     os.makedirs("backend/api/static", exist_ok=True)
70     app.mount("/static", StaticFiles(directory="backend/api/static"), name="static")
71
72     @app.get("/", response_class=HTMLResponse)
73     def serve_index():
74         index_path = "backend/api/static/index.html"
75         if os.path.exists(index_path):
76             with open(index_path, "r", encoding="utf-8") as f:
77                 return f.read()
78         return "<h3>Sports Highlight Indexer Server is Running. Please create index.html in
79         backend/api/static/</h3>"
80
81     # Global variables initialized at startup
82     db_instance: Optional[Database] = None
83     global_config: Optional[AppConfig] = None # now typed (Pydantic model)
84
85     # Async job worker ? drain/wake loop (DEFERRED_HARDENING_PLAN #16, EXECUTION_POLICY.md
86     # P3)
87     # now lives in backend.api.job_worker. Re-exported here so the names stay addressable on
88     # `backend.main` (`JobWaiting`, `_drain_due_jobs`, `_arm_wake_timer`, `_get_executor`,
89     # `_run_claimed_job`, `_MAX_DRAIN_WAKE_SEC`, ...). The worker resolves these seams
90     # THROUGH
91     # this module object at call-time, so `monkeypatch.setattr(backend.main, ...)` still
92     # intercepts the inter-function calls (a parked job arms the PATCHED `_arm_wake_timer`,
93     # so no real daemon timer leaks in tests). Behavior is unchanged by the extraction.
94     from backend.api.job_worker import ( # noqa: F401 (re-exports: names stay addressable
95     on backend.main)
96         JobWaiting,
97         _arm_wake_timer,
98         _drain_due_jobs,
99         _get_executor,
100         _job_to_request,
101         _MAX_DRAIN_WAKE_SEC,
102         _maybe_record_job_cost,
103         _run_claimed_job,
104         _schedule_next_drain_if_waiting,
105     )
106
107     # Legacy thin wrapper for any remaining direct calls during transition.

```

Now let me investigate the import-time CORS load concern more deeply. Let me check what `load\_config` does, whether it reads a file, and whether `global\_config` is set separately at startup (so the import-time load is redundant / potentially divergent).

13. user (2026-06-21T13:40:32.243Z)

```
diff --git a/backend/config/models.py b/backend/config/models.py
index 71cdc0e..4f27623 100644
--- a/backend/config/models.py
+++ b/backend/config/models.py
@@ -293,6 +293,20 @@ class SecurityConfig(BaseModel):
===load_config===
backend/config/loader.py:102:def load_config(path: str | Path | None = None) -> AppConfig:
===global_config assignment===
78:global_config: Optional[AppConfig] = None # now typed (Pydantic model)
140:     return global_config
147:     sport = getattr(global_config, "sport", None)
148:     indexing = getattr(global_config, "indexing", None)
174:     local_video = storage.acquire_local_input(req.video_path, getattr(global_config,
"storage", None))
181:     val_results, val_context = registry.run_all_with_context(local_video, global_config)
185:     default_seg = getattr(getattr(global_config, "indexing", None), "default_segmenter",
"motion")
227:         segmenter = segmenter_cls(db_instance, global_config)
266:         db=db_instance, video_id=video_id, video_path=req.video_path,
config=global_config,
288:         owner_id = edge_auth.resolve_tenant(request.headers, global_config)
292:     allowed_root = getattr(getattr(global_config, "security", None), "allowed_video_root",
None)
297:         input_ref = storage.resolve_input_ref(req.video_path, getattr(global_config,
"storage", None))
334:     fx = global_config.billing.fx_inr_per_usd
346:     fx = global_config.billing.fx_inr_per_usd
376:# backend.main.global_config, so behavior ? route paths, status codes, body-size limits,
396:     output_dir = getattr(getattr(global_config, "output", None), "output_dir", "output") or
"output"
444:     stitching = getattr(global_config, "stitching", None) or StitchingConfig()
467:     output_dir = getattr(getattr(global_config, "output", None), "output_dir", "output") or
"output"
476:         local_src = storage.acquire_local_input(video["filepath"], getattr(global_config,
"storage", None))
622:         # global_config isn't initialized this early; load the typed config so the trim
643:         global global_config, db_instance
644:         global_config = load_app_config() # returns typed AppConfig
645:         _enforce_startup_or_exit(global_config) # M5: fail fast on an incoherent ACTIVE profile
650:         global_config.output.output_dir = args.output_dir
651:         os.makedirs(global_config.output.output_dir, exist_ok=True)
653:         db_path = os.path.join(global_config.output.output_dir, global_config.output.db_name)
668:         storage_cfg = getattr(global_config, "storage", None)
684:         val_results, val_context = registry.run_all_with_context(local_video, global_config)
691:         seg_name = args.segmenter or getattr(getattr(global_config, "indexing", None),
"default_segmenter", "motion")
716:         segmenter = segmenter_cls(db_instance, global_config)
732:         db=db_instance, video_id=video_id, video_path=args.video_path,
config=global_config,
754:         # shared mutable state (`db_instance`, `global_config`) via `import backend.main` ? a
SEPARATE
```

14. assistant (2026-06-21T13:40:35.024Z)

Now I understand the architecture. Let me look at `load\_config` (loader.py) to confirm it reads a file from cwd, and check the existing tests around config loading and main import to assess the import-time CORS concern.

15. user (2026-06-21T13:40:35.788Z)

```
95         """Return a plain dict of the current built-in defaults.
96
97         The AppConfig model is the single source of truth for defaults.
98         """
99         return AppConfig().model_dump(mode="json")
100
101
102     def load_config(path: str | Path | None = None) -> AppConfig:
103         """Load configuration with the following precedence:
104
105         1. Built-in defaults defined in the Pydantic models.
106         2. Values from the JSON file (if present and readable).
107         3. (Future) environment / CLI overrides.
108
109         Missing file or unreadable file ? returns a fully defaulted AppConfig.
110         Unknown keys in the file are tolerated (extra="allow" on the model).
111         """
112         cfg_path = Path(path) if path is not None else DEFAULT_CONFIG_PATH
113
114         file_data: dict[str, Any] = {}
115         if cfg_path.exists():
116             try:
117                 with open(cfg_path, "r", encoding="utf-8") as f:
118                     file_data = json.load(f)
119             except Exception as exc:
120                 # Non-fatal: continue with defaults + whatever we could parse.
121                 # In production we might log here.
122                 logger.warning(f"Failed to read {cfg_path}: {exc}. Using defaults + partial
data.")
123
124                 # A config.json that is valid JSON but not an OBJECT (e.g. `[ ]`, `42`, `"x"`,
`null`) would
125                 # otherwise crash startup: a truthy non-mapping hits `.items()` in
_unknown_key_paths, and any
126                 # non-mapping breaks the `{**defaults, **file_data}` unpack. Degrade to defaults +
warn, per the
127                 # loader's "bad input is non-fatal" contract.
128                 if not isinstance(file_data, dict):
129                     logger.warning("%s is not a JSON object (got %s); ignoring it and using
defaults.",
130                                   cfg_path, type(file_data).__name__)
131                 file_data = {}
132
133         if file_data:
134             unknown = _unknown_key_paths(file_data, AppConfig)
135             if unknown:
136                 logger.warning(
137                     "[CONFIG WARNING] %s contains keys the typed config does not "
138                     "recognize (typo'd or removed knobs ? top-level extras are kept "
139                     "but unread; unknown section keys are silently dropped): %s",
140                     cfg_path, ", ".join(sorted(unknown)),
141                 )
142
143         # Precedence: built-in defaults < profile preset < config.json (< env/--set, applied
at
144         # consumption sites downstream). A *deployment profile* is opt-in ? it is applied
ONLY when
145         # explicitly selected (DEPLOYMENT_PROFILE env or config.json deployment.profile).
With no
146         # profile the merge is byte-identical to the pre-M1 loader, so the default path is
unchanged.
147         defaults = _get_builtin_defaults()
148         profile = _resolve_explicit_profile(file_data)
```

```

149
150     if profile and not is_known_profile(profile):
151         # Only an unrecognised value from config.json reaches here (env typos already
fell back
152         # in _resolve_explicit_profile). Warn + apply no preset; the bad value also hits
the
153         # typed Literal at model_validate below, a hard clear error ? loud, never
silent.
154         logger.warning(
155             "[CONFIG] unrecognized deployment profile %r (valid: %s); applying no
preset.",
156             profile, ", ".join(PROFILE_PRESETS),
157         )
158         profile = ""
159
160     if profile:
161         merged = _deep_merge(_deep_merge(defaults, preset_for(profile)), file_data)
162         # Record the profile actually applied ? the env may have chosen it over
config.json,
163         # so re-stamp it onto the merged deployment section to keep the record honest.
164         merged.setdefault("deployment", {})

```

16. assistant (2026-06-21T13:40:38.909Z)

Now I have a clear picture. Let me check the `DEFAULT\_CONFIG\_PATH` and whether there's a `DEPLOYMENT\_PROFILE` env var or config.json that could change CORS at import-time, and whether the import-time load could trigger profile/env side effects.

17. user (2026-06-21T13:40:42.118Z)

```

30:DEFAULT_CONFIG_PATH = Path("config.json")
47:def _resolve_explicit_profile(file_data: dict[str, Any]) -> str:
52:     An *unrecognised* ``DEPLOYMENT_PROFILE`` env value warns and FALLS THROUGH to the file's
56:     env_profile = os.environ.get("DEPLOYMENT_PROFILE")
62:     "[CONFIG] unrecognized DEPLOYMENT_PROFILE env value %r (valid: %s); ignoring it "
112:     cfg_path = Path(path) if path is not None else DEFAULT_CONFIG_PATH
145:     # explicitly selected (DEPLOYMENT_PROFILE env or config.json deployment.profile). With
no
148:     profile = _resolve_explicit_profile(file_data)
152:     # in _resolve_explicit_profile). Warn + apply no preset; the bad value also hits the
189:     "set deployment.profile or DEPLOYMENT_PROFILE to apply (auto-detect never "
202:     cfg_path = Path(path) if path is not None else DEFAULT_CONFIG_PATH
===config.json in repo?===
-rw-r--r-- 1 avidu 197609 2735 Jun 16 01:52 config.json
===existing import of backend.main in tests===
tests/test_api_endpoints.py
tests/test_async_jobs.py
tests/test_billing.py
tests/test_config.py
tests/test_edge_auth.py
tests/test_import_smoke.py
tests/test_job_worker_loop.py
tests/test_job_worker_module_seam.py
tests/test_server_entrypoint.py
tests/test_startup_checks.py

```

18. assistant (2026-06-21T13:40:46.637Z)

Important finding: `DEFAULT\_CONFIG\_PATH = Path("config.json")` is a **relative path** resolved against the current working directory, and there IS a `config.json` at the repo root. This means the import-time `load\_app\_config()` reads whatever `config.json` exists in cwd. Let me check the repo's `config.json` to see whether it sets `cors\_allow\_origins` or `deployment.profile` (which would change behaviour), and confirm the import-time read doesn't already happen elsewhere.

19. user (2026-06-21T13:40:46.924Z)

```
1      {
2          "sport": "badminton",
3          "ai_provider": "gemini",
4          "ai_model": "gemini-flash-latest",
5          "validation": {
6              "min_resolution_width": 1280,
7              "min_resolution_height": 720,
8              "min_fps": 29.9,
9              "min_blur_threshold": 20.0,
10             "max_scene_cuts_per_minute": 0.5,
11             "relevance": {
12                 "court_green_lower": [
13                     35,
14                     40,
15                     40
16                 ],
17                 "court_green_upper": [
18                     85,
19                     255,
20                     255
21                 ],
22                 "court_pixels_min_ratio": 0.05
23             }
24         },
25         "indexing": {
26             "default_segmenter": "gemini",
27             "use_gpu": true,
28             "motion_threshold": 0.08,
29             "min_segment_duration": 3.0,
30             "dead_time_merge_gap": 2.0,
31             "chunk_duration": 90,
32             "chunk_overlap": 10,
33             "detector_model": "fasterrcnn_resnet50_fpn",
34             "detector_score_threshold": 0.5,
35             "yolo_velocity_threshold": 0.15,
36             "yolo_frame_skip": 3,
37             "yolo_tracker": "bytetrack.yaml",
38             "yolo_prefetch_workers": 2,
39             "min_action_window_duration": 2.0,
40             "log_yolo_candidates": true,
41             "store_yolo_candidates": true,
42             "ai_handoff_padding": 2.0,
43             "ai_max_workers": 1,
44             "tracknet": {
45                 "wsl_distro": "Ubuntu",
46                 "repo_dir": "~/models/TrackNetV4",
47                 "conda_sh": "~/miniconda3/etc/profile.d/conda.sh",
48                 "conda_env": "TrackNetV4",
49                 "weights_path": "",
50                 "queue_length": 5,
51                 "stage_in_wsl": true,
52                 "wsl_stage_dir": "~/clips",
53                 "timeout_sec": 1800,
54                 "velocity_threshold": 0.02
55             },
56             "wasb": {
57                 "wsl_distro": "Ubuntu",
58                 "repo_dir": "~/models/WASB-SBDT",
59                 "conda_sh": "~/miniconda3/etc/profile.d/conda.sh",
60                 "conda_env": "wasb",
61                 "weights_path": "~/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar",
62                 "sport": "badminton",
```

```

63         "wsl_stage_dir": "~/clips",
64         "timeout_sec": 1800,
65         "velocity_threshold": 0.02
66     }
67 },
68     "output": {
69         "default_highlights_name": "highlights.mp4",
70         "db_name": "sports_indexer.db"
71     },
72     "stitching": {
73         "begin_padding": 0.5,
74         "end_padding": 0.5,
75         "use_cache": false,
76         "min_shot_count": 1,
77         "watermark": {
78             "enabled": true,
79             "text": "Generated by Khelsutra.guru",
80             "position": "bottom-right",
81             "opacity": 0.85,
82             "logo_path": "",
83             "font_path": ""
84         }
85     }
86 }

```

20. assistant (2026-06-21T13:40:51.944Z)

The repo `config.json` has no `security` section, so `cors\_allow\_origins` defaults to `["\*"]` ? no behaviour change for the repo's own config. The import-time read is bounded (no torch, no network). Now let me verify the most important behavioural concern: does `import backend.main` already trigger a config read at module level on master, or is this NEW import-time I/O? Let me check the master version of main.py around the CORS block.

21. user (2026-06-21T13:40:54.912Z)

```

=== master main.py module-level load_app_config calls ===
33:     load_config as load_app_config,
93:# New code should import load_app_config / AppConfig from backend.config directly.
94:def load_config(config_path: str = "config.json") -> Dict[str, Any]:
95:     cfg = load_app_config(config_path)
477:     cfg = load_config() # legacy wrapper still works, returns dict for now
610:         stitching = load_app_config().stitching
629:     global_config = load_app_config() # returns typed AppConfig
=== Does master call load at import (before any def)? checking lines 40-70 ===
app = FastAPI(title="Sports Highlight Indexer API")

```

Enable CORS for the local web interface. `allow_origins='*' with allow_credentials=True rejected by browsers per the fetch spec and is an unsafe default to carry into the auth/tenancy work; this tool uses no cookies, so credentials stay off.`

```

app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_credentials=False,
    allow_methods=["*"],
    allow_headers=["*"],
)

```

Serves static frontend files directly (now under api/ per approved structure)

```

os.makedirs("backend/api/static", exist_ok=True)
app.mount("/static", StaticFiles(directory="backend/api/static"), name="static")

@app.get("/", response_class=HTMLResponse)

```

```
def serve_index():
```

22. assistant (2026-06-21T13:41:01.453Z)

Confirmed: on master, `import backend.main` did NOT read `config.json` at module import (all `load\_app\_config()` calls were inside functions). This change introduces NEW import-time config I/O. This is a real behaviour change worth flagging, though bounded. Let me assess the test-reconfigurability concern: tests set `m.global\_config` via monkeypatch, but the CORS middleware is now frozen at import from a *different* config than `global\_config`. Let me check the mypy CI lane and whether the lazy `import jwt` passes mypy with the fixed package list.

23. user (2026-06-21T13:41:03.274Z)

```
=== CI workflow files ===
ci.yml
=== mypy lane: how deps installed ===
.github/workflows/ci.yml:29:      run: python -m pip install ruff
.github/workflows/ci.yml:33:      # Type gate (mypy.ini at repo root): lenient green baseline with
an explicit
.github/workflows/ci.yml:35:      # up. Typed core deps are installed so mypy sees their real
signatures
.github/workflows/ci.yml:45:      - name: Install mypy + typed core deps
.github/workflows/ci.yml:46:      run: python -m pip install mypy pydantic fastapi uvicorn
python-dotenv google-genai numpy
.github/workflows/ci.yml:48:      run: mypy backend training
.github/workflows/ci.yml:63:      run: python -m pip install pytest fastapi uvicorn pydantic
python-dotenv google-genai
.github/workflows/ci.yml:84:      run: python -m pip install torch torchvision --index-url
https://download.pytorch.org/whl/cpu
.github/workflows/ci.yml:87:      # (pytest, pytest-cov, httpx for the fastapi TestClient,
ruff, mypy).
.github/workflows/ci.yml:88:      run: python -m pip install -e .[dev]
.github/workflows/ci.yml:118:     run: python -m pip install pytest fastapi uvicorn pydantic
python-dotenv google-genai numpy
```

24. assistant (2026-06-21T13:41:05.637Z)

This is a key finding. Let me read the mypy lane fully and check `mypy.ini` for how it handles missing imports.

25. user (2026-06-21T13:41:06.587Z)

```
33      # Type gate (mypy.ini at repo root): lenient green baseline with an explicit
34      # per-module quarantine list ? ratchet by removing entries as modules clean
35      # up. Typed core deps are installed so mypy sees their real signatures
36      # (without them ignore_missing_imports would Any-stub half the checks).
37      typecheck:
38        name: Mypy (lenient baseline)
39        runs-on: ubuntu-latest
40        steps:
41          - uses: actions/checkout@v6
42          - uses: actions/setup-python@v6
43            with:
44              python-version: "3.13"
45          - name: Install mypy + typed core deps
46            run: python -m pip install mypy pydantic fastapi uvicorn python-dotenv google-
genai numpy
47          - name: Mypy
48            run: mypy backend training
49
50      # Fast guard against the 7fbb0 class of breakage: a SyntaxError or
51      # module-level import error shipping while the suite stays green.
52      # Deliberately does NOT install cv2/torch/numpy ? the smoke tests must hold
53      # on machines without the GPU stack (test_import_smoke stubs what's absent).
```

26. user (2026-06-21T13:41:08.421Z)

Exit code 2

=== mpy.ini ===

[mpy]

Lenient, GREEN baseline (quality sweep AI-4): the gate protects the clean modules and pins an explicit quarantine list below ? not a typing crusade.

Ratchet: when you touch a quarantined module, try removing its entry.

ignore_missing_imports = True

backend/ is a namespace package (no root __init__.py) ? required so mpy maps files to backend.* module names instead of duplicating top-level names.

explicit_package_bases = True

--- Quarantine (known-loose modules, each with a reason) ---

Optional module globals (db_instance/global_config set only in main()); properly fixed by the deferred app-factory/lifespan refactor (BACKLOG).

[mpy-backend.main]

ignore_errors = True

Job-worker drain/wake loop extracted from backend.main (same Optional module-glob debt: resolves db_instance/global_config THROUGH backend.main at call-time). Shares backend.main's quarantine until the deferred app-factory/lifespan refactor lands.

[mpy-backend.api.job_worker]

ignore_errors = True

Worker dispatcher: composes the SAME validator/segmenter/report building blocks as backend.main, which are typed `config: dict` but receive the AppConfig migration shim at runtime (the .get() compat layer). Shares backend.main's config-shim quarantine until the typed-config migration removes the dict?AppConfig duality everywhere.

[mpy-backend.worker.__main__]

ignore_errors = True

Legacy demo segmenter (fabricated metadata; coverage-omitted too).

[mpy-backend.pipeline.segmenters.motion]

ignore_errors = True

yolo_hybrid: the fusion-P1 extraction moved its loose torchvision/cv2 inline code to detectors/person_detector.py (which IS type-clean, no quarantine). What remains here is pre-existing orchestration debt unrelated to that move (implicit-Optional process_video defaults, the Success|Failure ABC-override mismatch, a Future[...] annotation) ? ratchet those out in a focused typing pass, not here.

(wasb_hybrid/tracknet_hybrid left the quarantine with the AI-7 collapse.)

[mpy-backend.pipeline.segmenters.yolo_hybrid]

ignore_errors = True

[mpy-backend.pipeline.segmenters.gemini]

```
ignore_errors = True
```

Registry builds cls(None, {}) probes; typing it honestly needs a Protocol.

```
[mypy-backend.pipeline.segmenters.base]
ignore_errors = True
=== ignore_missing_imports? ===
mypy.ini:5:ignore_missing_imports = True
```

27. assistant (2026-06-21T13:41:14.821Z)

This resolves the mypy concern: `ignore\_missing\_imports = True` is set globally in `mypy.ini`, so the lazy `import jwt` / `from jwt import PyJWTSet` in `backend/api/auth.py` will NOT fail mypy even though PyJWT isn't installed in the mypy lane. The lazy import inside the function is fine either way. So lens item (5) is NOT a real issue.

Now let me verify the schema actually has `owner\_id` available for fresh DBs (v6 migration runs on init) so `create\_job` with the new column won't fail on a fresh test DB, and confirm `get\_job` returns `owner\_id`. Let me also confirm `\_add\_column\_idempotent` / that a brand-new DB reaches user_version 6.

28. user (2026-06-21T13:41:17.054Z)

```
69:         ``user_version`` stays un-bumped ? and the next open would re-run the same
88:         # Versioned migrations live here. PRAGMA user_version is the schema
91:         version = cursor.execute("PRAGMA user_version").fetchone()[0]
109:         self._migrate_to_v6(cursor)
110:         # future: if version < 7: cursor.execute("ALTER TABLE ..."); PRAGMA user_version
= 7
183:         cursor.execute("PRAGMA user_version = 1")
210:         cursor.execute("PRAGMA user_version = 2")
222:         cursor.execute("PRAGMA user_version = 3")
242:         cursor.execute("PRAGMA user_version = 4")
280:         CREATE UNIQUE INDEX IF NOT EXISTS idx_user_models_user_version
290:         cursor.execute("PRAGMA user_version = 5")
292:     def _migrate_to_v6(self, cursor):
337:         cursor.execute("PRAGMA user_version = 6")
592:     def get_job(self, job_id: str) -> Optional[Dict[str, Any]]:
```

29. assistant (2026-06-21T13:41:21.820Z)

The migration ladder reaches v6 on a fresh DB (line 109), so `owner\_id` column exists. `get\_job` returns it (test asserts `["owner\_id"]`). Now let me run the new test file and a couple of the broader job/api tests to confirm nothing regresses, and verify the test for default-OFF actually passes given the import-time CORS quirk.

30. user (2026-06-21T13:41:30.045Z)

```
PyJWT 2.13.0
=== run edge auth tests ===
..... [100%]
17 passed in 5.05s
```

31. assistant (2026-06-21T13:41:34.164Z)

All 17 edge-auth tests pass. Now let me check whether the import-time CORS load could actually break or surprise the existing `test\_import\_smoke.py` (which deliberately runs without torch/cv2 and stubs absent modules) ? does it monkeypatch cwd or assume no config read? And run the import-smoke + a sample of api/job tests to confirm no regression from the new import-time I/O.

32. user (2026-06-21T13:42:05.380Z)

```
..... [100%]
49 passed in 28.11s
```

33. assistant (2026-06-21T13:42:09.155Z)

All green. Now let me trace one subtle integration concern: the ``verify_cf_access_token`` import does ``from jwt import PyJWKSet``, but the test imports ``from jwt.algorithms import RSAAlgorithm``. Let me verify ``PyJWKSet`` and ``PyJWKSet.from_dict(...)[kid]`` actually exist in the PyJWT API as used (the ``KeyError`` catch relies on ``__getitem__`` raising ``KeyError`` for unknown kid). Let me also double-check the ``signing_key.key`` attribute path.

34. user (2026-06-21T13:42:11.818Z)

```
PyJWKSet has from_dict: True
PyJWKSet has __getitem__: True
jwt.get_unverified_header: True
```

35. assistant (2026-06-21T13:42:17.659Z)

API surface confirmed. The unknown-kid path raises ``KeyError`` (caught explicitly), and the test ``{"kid": "unknown-kid"}`` passes, confirming it.

Now let me assess one final default-OFF subtlety on the ``resolve_tenant`` trace for item (1): with ``edge_auth_enabled=False``, the function checks ``getattr(sec, "edge_auth_enabled", False)`` and returns ``None`` immediately ? before ever touching the token or JWKS. The test ``test_resolve_tenant_off_is_noop_even_with_token`` confirms this with a valid token present. This is correct and grounded.

Let me do a final check on whether there are any OTHER HTTP callers/tests of ``/api/process`` that construct the endpoint signature in a way the new ``request: Request`` param could break (e.g. direct function calls ``process_video(req)`` rather than via ``TestClient``).

36. user (2026-06-21T13:42:19.560Z)

```
=== direct calls to process_video( ? ===
backend/main.py:229:         result = segmenter.process_video(video_id, local_video,
req.player_pool, req.max_frames)
backend/main.py:717:         result = segmenter.process_video(video_id, local_video,
max_frames=args.max_frames)
backend/results.py:72:     that forget it (e.g. ``len(process_video(...))``) crash with
``TypeError: ... has no len()``
backend/worker/__main__.py:206:         result = segmenter_cls(db,
config).process_video(video_id, local_video, max_frames=args.max_frames)
Binary file backend/__pycache__/results.cpython-313.pyc matches
tests/test_base.py:34:     success = inst.process_video("v1", "path")
tests/test_base.py:40:     failure = inst.process_video("v1", "fail")
tests/test_base.py:192:         inst.process_video("v1", "path")
tests/test_fusion_hybrid.py:55:         res = seg.process_video("v1", "clip.mp4")
tests/test_gemini.py:42:     res = segmenter.process_video("v1", "dummy.mp4")
tests/test_gemini.py:81:     res = segmenter.process_video("v1", "dummy.mp4")
tests/test_gemini.py:108:         return segmenter.process_video(video_id, "dummy.mp4")
tests/test_gemini.py:185:         res = segmenter.process_video("vid_single_oversize",
"dummy.mp4")
tests/test_motion_segmenter.py:114:         result = segmenter.process_video("vid-1", "bad.mp4")
tests/test_motion_segmenter.py:131:         result = segmenter.process_video("vid-1",
"quiet.mp4")
tests/test_motion_segmenter.py:162:         result = segmenter.process_video("vid-1",
"active.mp4")
tests/test_output_base.py:77:         seg.process_video("v1", "clip.mp4")
tests/test_profile_parity.py:69:     result = seg.process_video("vidParity", str(video))
tests/test_tracknet_hybrid.py:63:     res = seg.process_video("v1", "clip.mp4")
tests/test_tracknet_hybrid.py:82:     assert seg.process_video("v1", "clip.mp4") == []
tests/test_tracknet_hybrid.py:95:     res = seg.process_video("v1", "clip.mp4")
tests/test_tracknet_hybrid.py:110:     assert seg.process_video("v1", "clip.mp4") == []
tests/test_tracknet_hybrid.py:129:     seg.process_video("v1", "clip.mp4")
tests/test_tracknet_hybrid.py:153:     res = seg.process_video("v1", "clip.mp4")
tests/test_tracknet_hybrid.py:181:     res = seg.process_video("v1", "clip.mp4")
```

```

tests/test_trajectory_hybrid_equivalence.py:56:         return db, seg.process_video("v1",
"clip.mp4")
tests/test_yolo_hybrid.py:90:     res = segmenter.process_video("v1", "dummy.mp4")
tests/test_yolo_hybrid.py:114:    res = segmenter.process_video("v1", "dummy.mp4")
tests/test_yolo_hybrid.py:157:    res = segmenter.process_video("v1", "dummy.mp4")
tests/test_yolo_hybrid.py:226:    res = segmenter.process_video("v1", "dummy.mp4")
tests/test_yolo_hybrid.py:277:    segmenter.process_video("v1", "dummy.mp4")
tests/test_yolo_hybrid.py:314:    segmenter.process_video("v1", "dummy.mp4")
=== /api/process callers in tests ===
tests/test_api_endpoints.py:74:    r = client.post("/api/process", json={"video_path":
str(vid_file), "segmenter": "x"})
tests/test_api_endpoints.py:89:    r = client.post("/api/process", json={"video_path":
str(vid_file)})
tests/test_api_endpoints.py:100:   r = client.post("/api/process", json={"video_path":
"a\x00b.mp4"})
tests/test_api_endpoints.py:131:   r = client.post("/api/process", json={"video_path":
str(vid_file)})
tests/test_api_endpoints.py:163:   r = client.post("/api/process", json={"video_path":
str(vid_file)})
tests/test_api_endpoints.py:175:   r2 = client.post("/api/process", json={"video_path":
str(vid_file)})
tests/test_api_endpoints.py:360:   rj = client.post("/api/process", json={"video_path":
str(vid_file), "segmenter": "x"})
tests/test_api_endpoints.py:363:   rd = client.post("/api/process", json={"video_path":
str(vid_file), "segmenter": "x"})
tests/test_api_endpoints.py:370:   rf = client.post("/api/process", json={"video_path":
str(vid_file)})
tests/test_api_endpoints.py:398:   r = client.post("/api/process", json={"video_path":
str(vid), "segmenter": "x"})

```

37. assistant (2026-06-21T13:42:26.159Z)

The `process\_video` matches are all the segmenter method (different function), not the endpoint. All `/api/process` callers go through `client.post` (TestClient), where FastAPI injects `Request` automatically? `request: Request` is a non-body param, so it does NOT alter the OpenAPI request body schema or break existing callers. The api_endpoints tests passed above, confirming this. Lens item (4) is not an issue.

I have thoroughly traced all five lens items. Let me compile findings. The substantive real finding is the new import-time config I/O in `main.py` (NEW behaviour vs master, plus the test-reconfigurability gap where CORS is fixed at import from a possibly-different config than the monkeypatched `global\_config`). The rest of the lens items resolve to non-issues, grounded by tests and code.

Let me do one last grounded check on the test-reconfigurability point: confirm there's no existing CORS test that monkeypatches `global\_config` and expects CORS to reflect it.

38. user (2026-06-21T13:42:28.448Z)

```
=== EXIT (none means no CORS test) ===
```

39. assistant (2026-06-21T13:42:34.377Z)

No CORS tests exist, so the import-time freeze doesn't break any current test, but it does mean the new `cors\_allow\_origins` knob is effectively untested and cannot be exercised per-test via the existing `monkeypatch.setattr(m, "global\_config", ...)` pattern (the middleware is baked at import). That's a real, if minor, observability/testability gap worth noting.

I have completed the review. Let me submit findings.

40. user (2026-06-21T13:42:50.766Z)

Structured output provided successfully

41. assistant (2026-06-21T13:43:02.386Z)

Review complete. I traced all five lens items against the actual code and ran the relevant tests.

Summary of what I found (full detail in the structured output):

Two real, minor findings:

1. `backend/main.py:52-55` ? the import-time `load_app_config()` for CORS is genuinely new behaviour vs master (which never touched the filesystem on `import backend.main`). It's bounded and safe for this repo (torch-free, network-free, `try/except`, and the repo's `config.json` has no `security` section), but it now does cwd-relative, env-sensitive config I/O on every import, including the `DEPLOYMENT_PROFILE` env read inside `_resolve_explicit_profile`.
2. `backend/main.py:52-62` ? the CORS middleware is frozen at import from a separate config load than the monkeypatched `global_config`, so the new `cors_allow_origins` knob is untestable via the existing seam and ships with zero coverage.

Lens items that resolved to NON-issues (grounded):

- `(1) default-OFF trace`: `resolve_tenant` returns `None` before touching token/JWKS when `edge_auth_enabled=False`; the test `test_resolve_tenant_off_is_noop_even_with_token` proves `None` even with a valid token present.
- `(3) create_job 6th param`: every caller (grep across `backend/` + `tests/`) passes at most 5 positional args then `policy=/'owner_id='` as keywords; no caller passes a 6th positional, so adding `owner_id` is safe. The `owner_id` column already exists from the M8/v6 migration (`database.py:300`), so no schema change.
- `(4) request: Request`: all `/api/process` callers go through `TestClient.post`; `Request` is a non-body dependency that doesn't alter the OpenAPI body schema; `test_api_endpoints.py` passes.
- `(5) mypy on lazy import jwt`: `mypy.ini` sets `ignore_missing_imports = True` globally, and `backend.main` is in the quarantine list anyway; the lazy import inside `verify_cf_access_token` won't fail the mypy lane even though PyJWT isn't in its fixed package list.

All 17 edge-auth tests pass, plus import-smoke/api/async-job suites (49 tests) green. PyJWT `PyJWKSet.from_dict(...)[kid] / .key` API surface verified against the installed PyJWT 2.13.0.